SANDIP UNIVERSITY
UGC Recognised

NAAC GRADE A

DEVSECOPS CI/CD PIPELINE

Automated Security Gates with GitHub Actions, Trivy, and SonarQube

Project Report

Objective: describe a CI/CD pipeline approach that integrates security scanning and code-quality analysis into automated GitHub Actions workflows, with explicit gates to prevent risky changes from progressing unnoticed.

Abstract

DevSecOps integrates security practices throughout the software delivery lifecycle. This project outlines a GitHub Actions pipeline using Trivy for vulnerability and configuration scanning and SonarQube/SonarCloud for code analysis. A companion Python utility checks local workflow YAML files for expected scanner and gate signals, and flags selected workflow risks such as excessive permissions, mutable action references, and non-blocking steps.

1. Problem statement

When security checks are performed only near release time, vulnerabilities and code-quality issues may be discovered late. A repeatable CI/CD workflow can run checks on changes and provide earlier feedback, while explicit gate behavior helps ensure that high-risk findings do not silently pass.

2. Objectives

• Integrate Trivy and SonarQube/SonarCloud into the GitHub Actions workflow design.
• Define security and quality checks as part of automated CI.
• Encourage least-privilege workflow permissions and safe secret handling.
• Provide a Python utility to inspect workflow configuration for key signals and risky patterns.
• Document execution, configuration, limitations, and recommended practices.

3. Architecture and workflow

1) A developer pushes code or opens a pull request. 2) GitHub Actions starts the workflow. 3) The repository is checked out with minimal permissions. 4) Trivy scans the selected scope (for example, filesystem, dependencies, container image, or IaC). 5) SonarQube/SonarCloud performs code analysis. 6) The configured vulnerability thresholds and quality gate determine whether the pipeline may proceed. 7) Findings are reviewed and remediated before release.

4. Tool roles

Component	Role in the pipeline
GitHub Actions	Runs automated jobs on pushes, pull requests, and approved release events.
Trivy	Scans selected source, dependency, container, or IaC targets for vulnerabilities and misconfiguration.
SonarQube / SonarCloud	Analyzes code quality and selected security rules; quality-gate enforcement must be configured.
Python workflow auditor	Reviews workflow YAML for scanner/gate signals and selected risky configuration patterns.

5. Python auditor implementation

The auditor searches for .yml/.yaml files beneath .github/workflows and checks for Trivy, SonarQube/SonarCloud, explicit gate signals, workflow permissions, pull_request_target, mutable action references, and continue-on-error settings. It prints findings with rule IDs, severity, file and line references, and recommendations. JSON output is available for reporting.

6. Example commands

```
python devsecops_pipeline_audit.py .
python devsecops_pipeline_audit.py . --json devsecops-audit.json
python devsecops_pipeline_audit.py . --fail-on MEDIUM
```

7. Implementation and operational recommendations

• Keep workflow permissions minimal and scoped to jobs that need them.
• Pin third-party actions to reviewed full commit SHAs where required.
• Use GitHub Secrets or OIDC-based short-lived credentials; never hard-code tokens.
• Configure Trivy scan scope, severity thresholds, and exit behavior deliberately.
• Configure SonarQube quality-gate waiting/enforcement so failed gates block release when required.
• Avoid running untrusted pull-request code with elevated permissions or secrets.
• Review scan results and document approved exceptions.

8. Limitations and future enhancements

The Python utility is a static text-based checker, not a workflow engine or security scanner. It may produce false positives or miss issues involving reusable workflows, expressions, composite actions, permissions inherited from organization settings, or external integrations. Future work could add YAML parsing, SARIF output, branch-protection integration, artifact retention, policy-as-code, and controlled end-to-end CI testing.

9. Conclusion

A well-designed DevSecOps pipeline makes security checks repeatable and visible during development. Combining GitHub Actions, Trivy, and SonarQube/SonarCloud can provide useful feedback and enforce defined thresholds when configured correctly. The included Python auditor adds a lightweight local review step, but the actual CI workflow and its security gates must be tested and maintained in the target repository.

Disclaimer: This is an educational project template. Review vendor documentation, action versions, and organization security policies before production use.